

Neural Networks

Machine Learning Crash Course

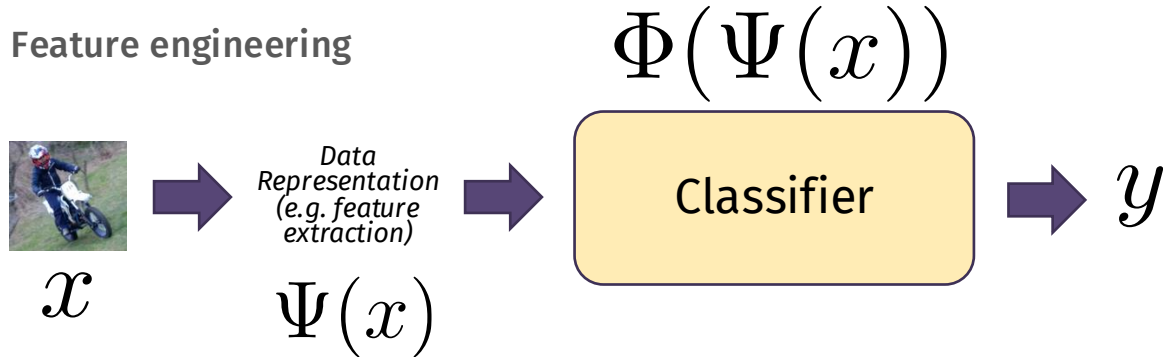
Nicoletta Noceti

Today

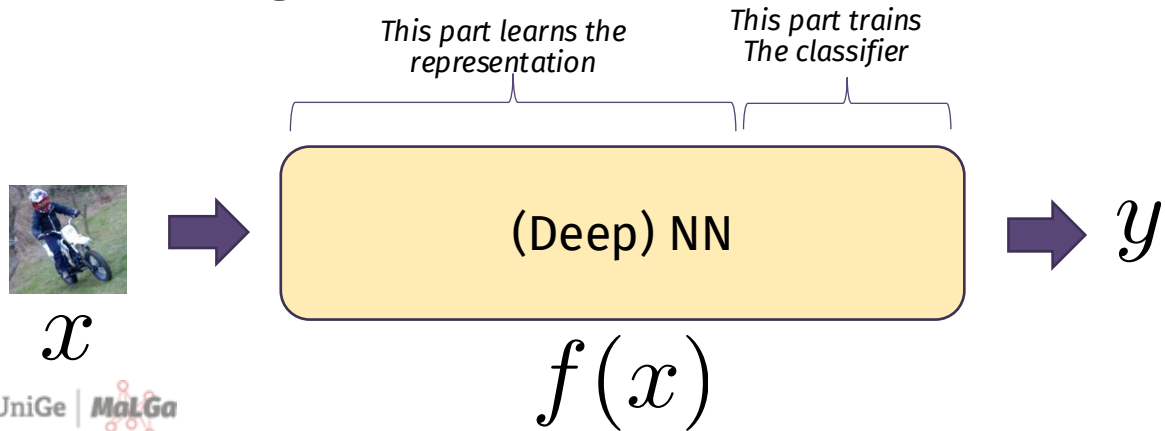
- We talk about a different family of learning algorithms, the Neural Networks
- We discuss how the concepts we know are adopted here

Neural Networks as representation learning

Feature engineering

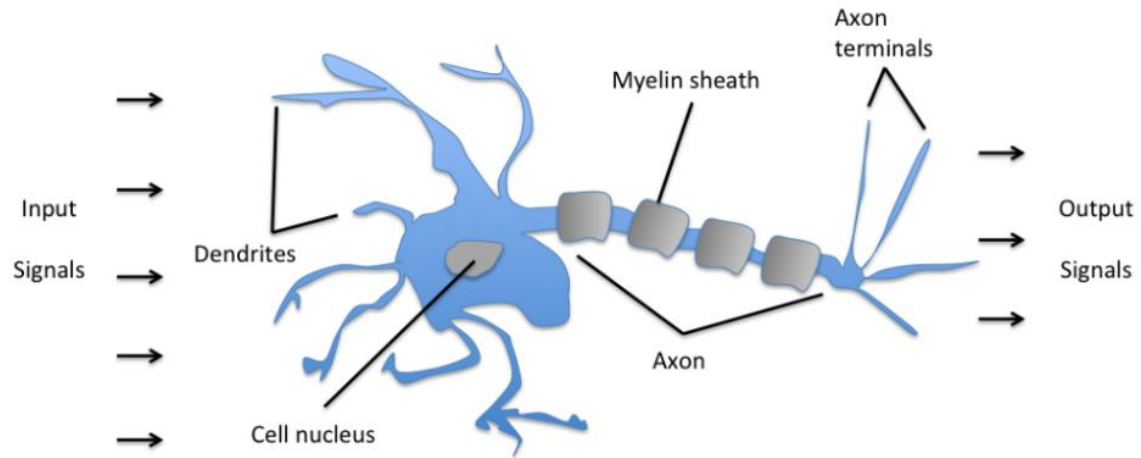


Feature learning

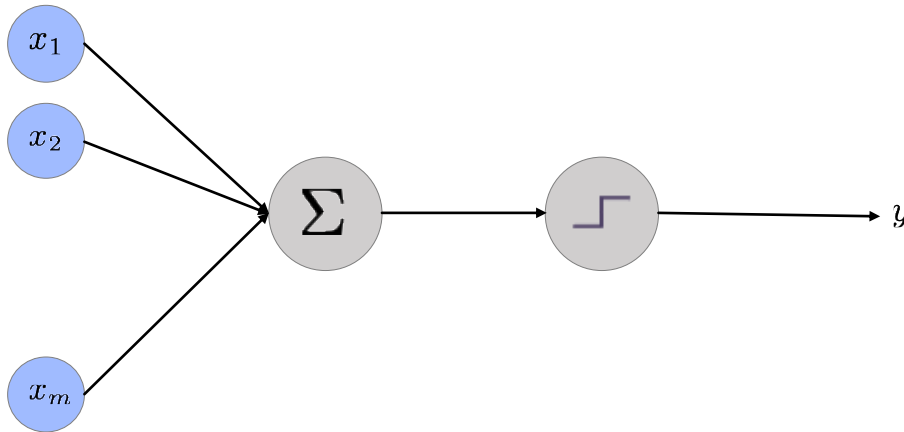


Single Layer Perceptron

Biological neuron



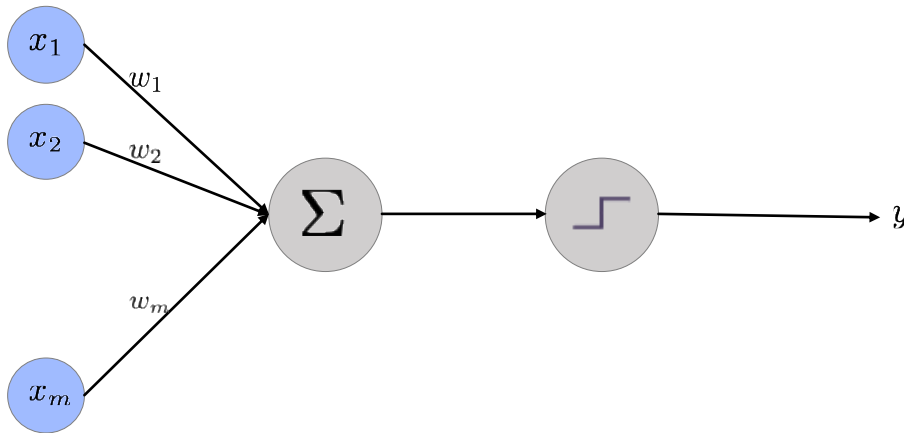
McCulloch & Pitts Neuron Model (1943)



$$f_{\theta} : \mathbb{R}^m \rightarrow \mathbb{R}$$

$$y = f_{\theta}(\mathbf{x}) = \begin{cases} 1 & \text{if } \sum_{i=1}^m x_i > \theta \\ 0 & \text{otherwise} \end{cases}$$

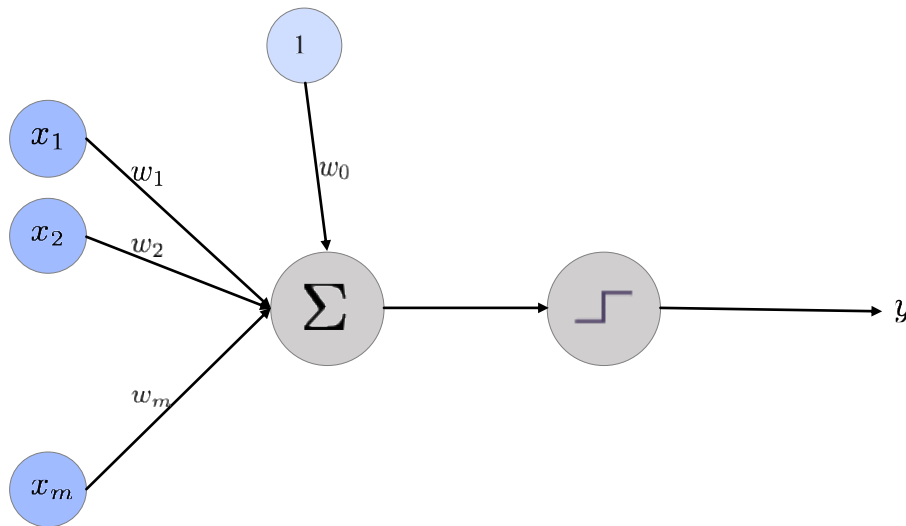
Towards the Single Layer Perceptron



$$f_{\mathbf{w},\theta} : \mathbb{R}^m \rightarrow \mathbb{R}$$

$$y = f_{\mathbf{w},\theta}(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{w}^\top \mathbf{x} > \theta \\ 0 & \text{otherwise} \end{cases}$$

Towards the Single Layer Perceptron



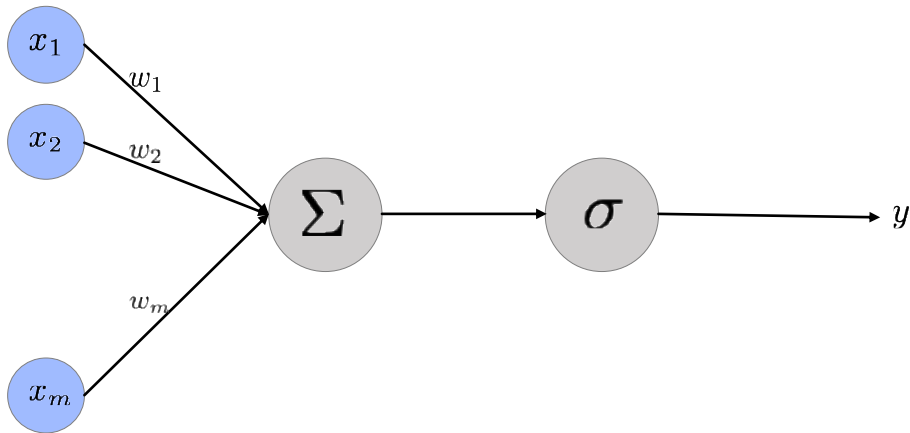
$$f_{\mathbf{w}, \theta} : \mathbb{R}^m \rightarrow \mathbb{R}$$

$$y = f_{\mathbf{w}, \theta}(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{w}^\top [1, \mathbf{x}] > \theta \\ 0 & \text{otherwise} \end{cases}$$

From now on we will use as standard the notation $\mathbf{w}^\top \mathbf{x}$

It means that we assume the presence of the bias term to be addressed

Towards the Single Layer Perceptron



$$f_{\mathbf{w}, \sigma} : \mathbb{R}^m \rightarrow \mathbb{R}$$

$$y = f_{\mathbf{w}, \sigma}(\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x})$$

Non-Linear activation functions

- Nonlinear activation is key to achieving good function approximation
- It takes a single number and maps it to a different numerical value
- Popular functions

Tanh

Sigmoid

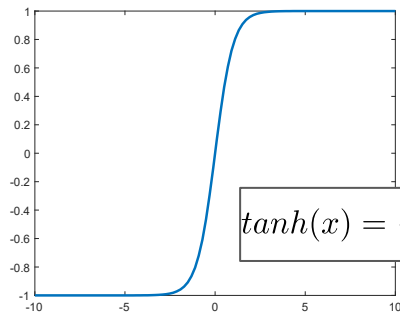
ReLU

Leak(y)
ReLU

Softmax

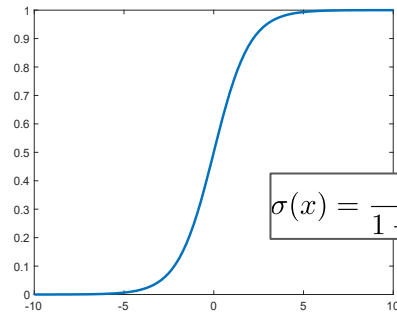
To practice with non-linear activation functions: <https://playground.tensorflow.org/>

Non-linear Activation functions



$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1$$

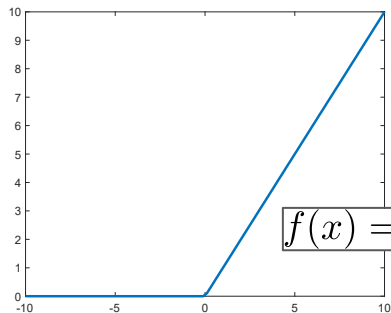
Tanh



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Sigmoid

ReLU

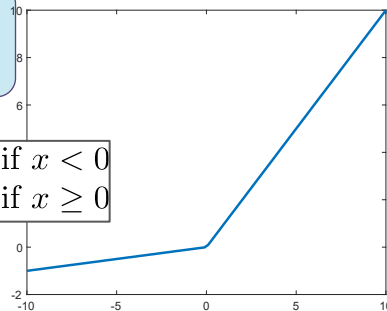


$$f(x) = \max(0, x)$$

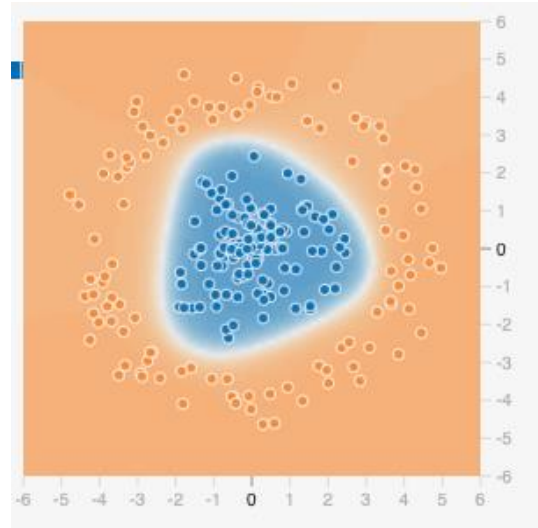
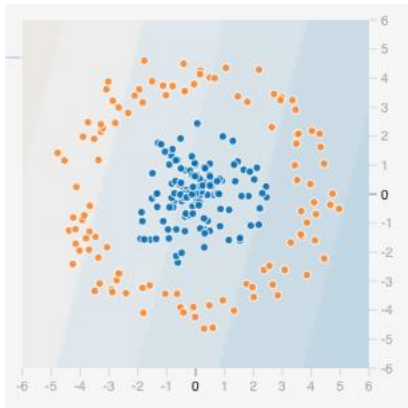
Leaky(y)
ReLU

$$f(x) = \begin{cases} \alpha x & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$

$$\alpha = 0.1$$

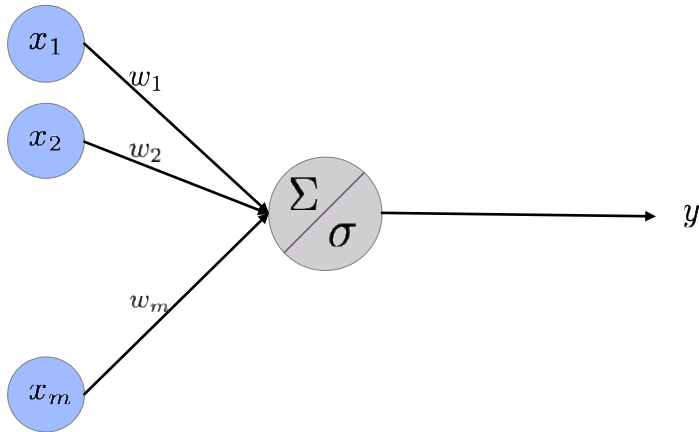


Non-linear Activation functions



To practice with non-linear activation functions: <https://playground.tensorflow.org/>

Towards the Single Layer Perceptron



$$f_{\mathbf{w},\sigma} : \mathbb{R}^m \rightarrow \mathbb{R}$$

$$y = f_{\mathbf{w},\sigma}(\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x})$$

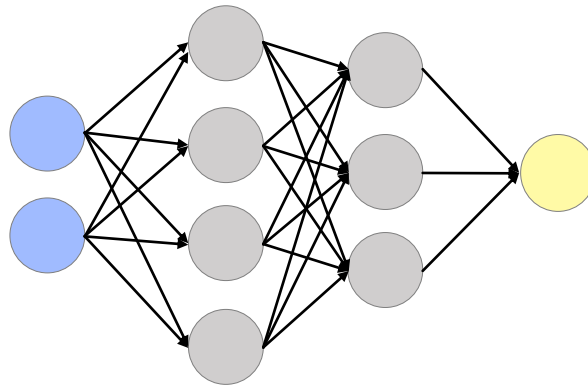
Multi-Layer Perceptron

Neural Networks

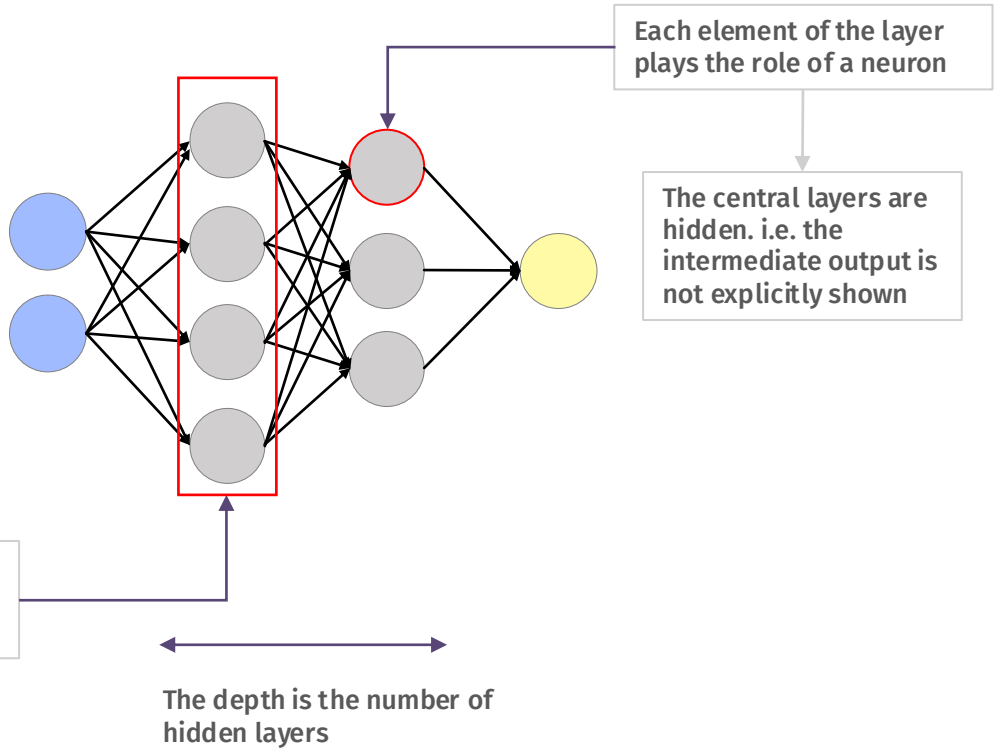
Neural Networks are composed of chains of layers that may be of three different types:

- INPUT LAYER
- (MULTIPLE) HIDDEN LAYER(S)
- OUTPUT LAYER

The layers are fully connected



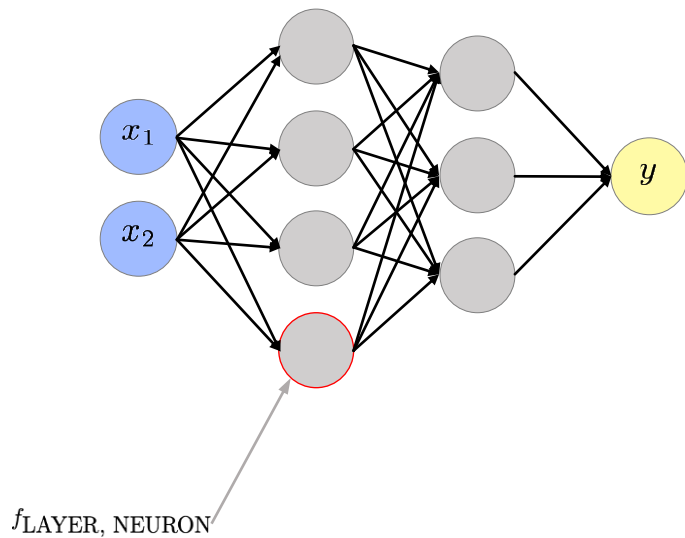
Neural Networks



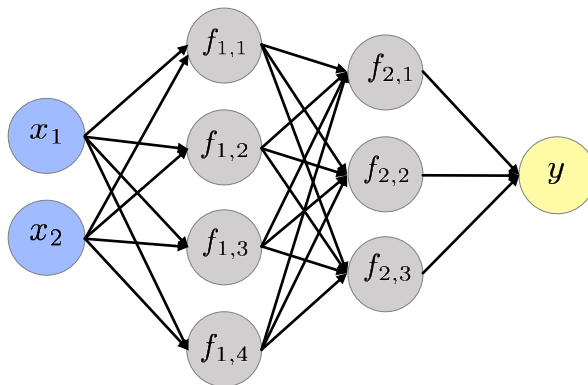
One network, two phases

- **Forward propagation:** the input flows into the network and produces a cost
- **Backward propagation:** allows the info to flow back into the net to compute the gradient (during the optimization)

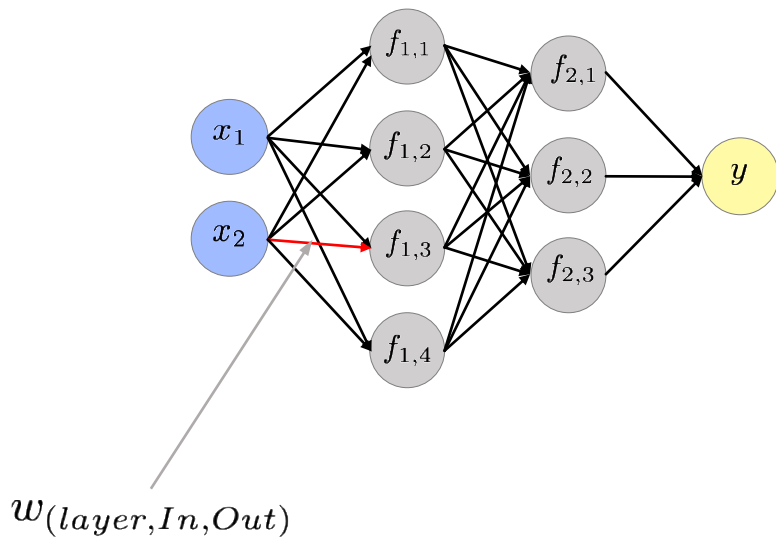
Forward propagation



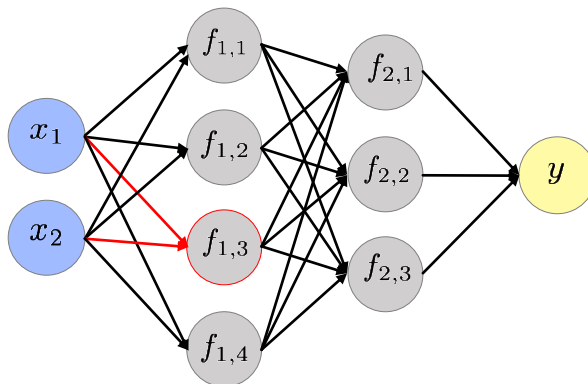
Forward propagation



Forward propagation



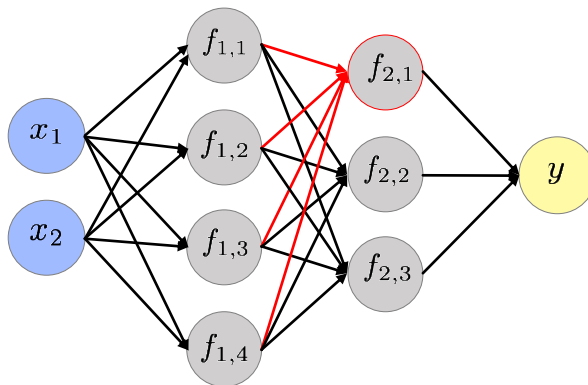
Forward propagation



$$f_{1,3}(\mathbf{x}) = \sigma(\mathbf{w}_{(1,:,3)}^\top \mathbf{x})$$

$$\mathbf{w}_{(1,:,3)} = [w_{(1,1,3)} w_{(1,2,3)}]$$

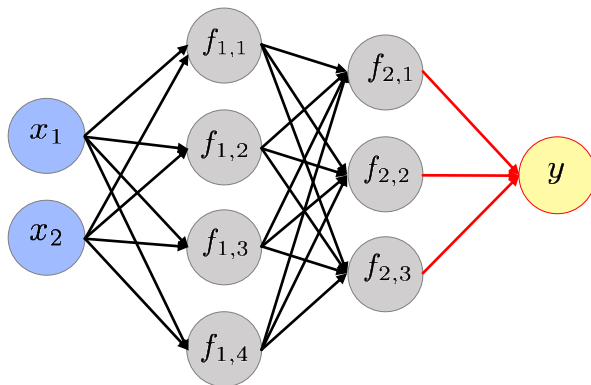
Forward propagation



$$f_{2,1}(\mathbf{f}_{(1,:)}) = \sigma(\mathbf{w}_{(2,:,1)}^\top \mathbf{f}_{(1,:)})$$

$$\mathbf{f}_{(1,:)} = [f_{1,1} f_{1,2} f_{1,3} f_{1,4}]$$

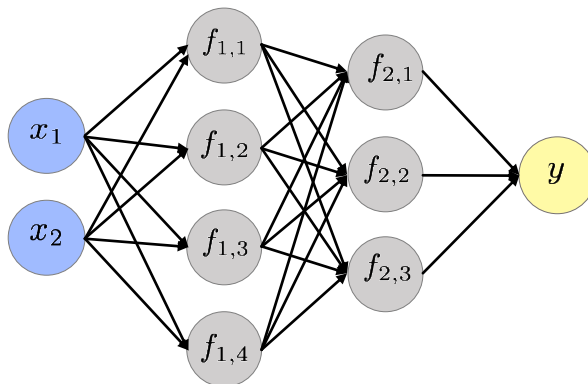
Forward propagation



$$y = \sigma(\mathbf{w}_{(o,:)}^T \mathbf{f}_{(2,:)})$$

$$\mathbf{f}_{(2,:)} = [f_{2,1} f_{2,2} f_{2,3}]$$

Forward propagation



... hence, In this example... $f_{\mathbf{w}} : \mathbb{R}^2 \rightarrow \mathbb{R}$

$$\mathbf{w} \in \mathbb{R}^{23+8}$$

weights

Bias
terms

Training a Neural Network

Training a Neural Network

- Training a (Deep) Neural Network means (as usual) learning the values for the model parameters (weights, bias terms) from the training set
- An essential element of training is (as usual) the loss function, which estimates how much we lose with the prediction in place of the real output y

$$\ell : Y \times Y \rightarrow [0, +\infty]$$

where Y is the space of the output of the estimated function

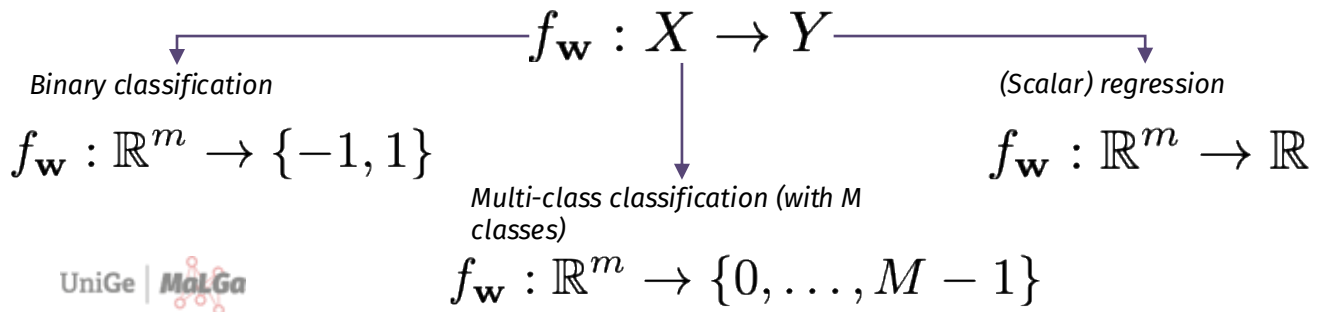
$$f_{\mathbf{w}} : X \rightarrow Y$$

Training a Neural Network

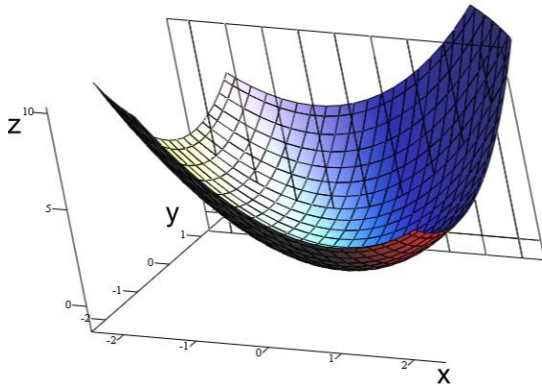
- Training a (Deep) Neural Network means (as usual) learning the values for the model parameters (weights, bias terms) from the training set
- An essential element of training is (as usual) the loss function, which estimates how much we lose with the prediction in place of the real output y

$$\ell : Y \times Y \rightarrow [0, +\infty]$$

where Y is the space of the output of the estimated function

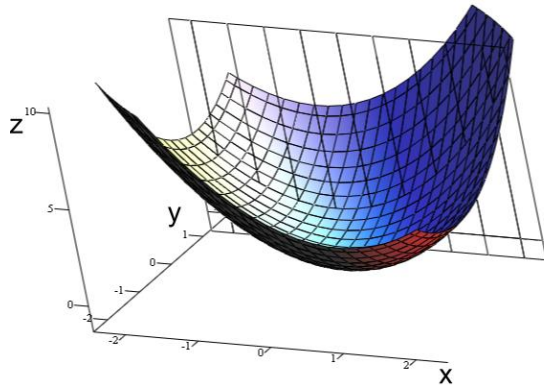


Training a deep NN is not...

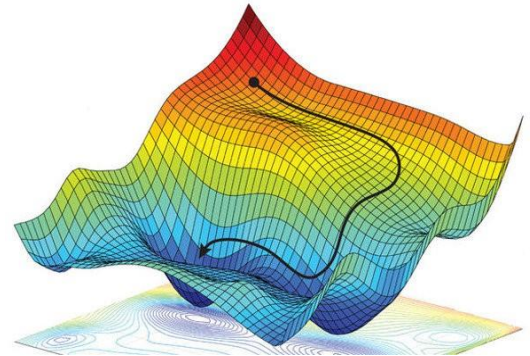


<https://towardsdatascience.com/understand-convexity-in-optimization-db87653bf920?gi=b93b0698a062>
<https://medium.com/swlh/non-convex-optimization-in-deep-learning-26fa30a2b2b3>

Training a deep NN is not...



...but more like...



<https://towardsdatascience.com/understand-convexity-in-optimization-db87653bf920?gi=b93b0698a062>
<https://medium.com/swlh/non-convex-optimization-in-deep-learning-26fa30a2b2b3>

Training a Neural Network

$$\mathbf{w}^* = \arg \min_{\mathbf{w} \in \mathbb{R}^P} \frac{1}{n} \sum_{i=1}^n \ell(f_{\mathbf{w}}(\mathbf{x}^i), y^i)$$

 Total number of model parameters

$$\mathbf{x}^i \in X$$

$$y^i \in Y$$

Training set $S = \{\mathbf{x}^i, y^i\}_{i=1}^n$

Training a Neural Network

$$\mathbf{w}^* = \arg \min_{\mathbf{w} \in \mathbb{R}^P} \frac{1}{n} \sum_{i=1}^n \ell(f_{\mathbf{w}}(\mathbf{x}^i), y^i) [+ \lambda \Psi(f_{\mathbf{w}})]$$

Total number of model parameters

$\mathbf{x}^i \in X$

$y^i \in Y$

Regularization term

Training set $S = \{\mathbf{x}^i, y^i\}_{i=1}^n$

Training a Neural Network

$$\mathbf{w}^* = \arg \min_{\mathbf{w} \in \mathbb{R}^P} \frac{1}{n} \sum_{i=1}^n \ell(f_{\mathbf{w}}(\mathbf{x}^i), y^i) [+ \lambda \Psi(f_{\mathbf{w}})]$$

 Total number of model parameters

$\mathbf{x}^i \in X$

$y^i \in Y$

 Regularization term

Training set $S = \{\mathbf{x}^i, y^i\}_{i=1}^n$

$$\mathbf{w}^* = \arg \min_{\mathbf{w} \in \mathbb{R}^P} J(S; \mathbf{w})$$

To find the parameters we need
an optimization strategy

Training a Neural Network

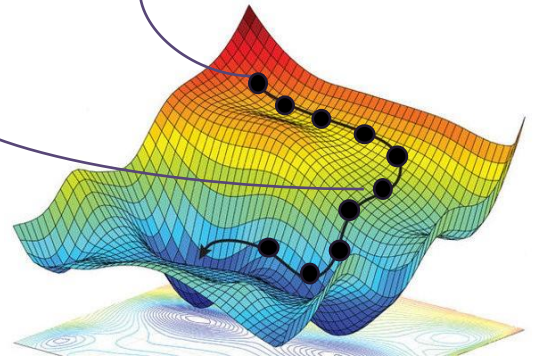
$$\mathbf{w}^* = \arg \min_{\mathbf{w} \in \mathbb{R}^P} J(S; \mathbf{w})$$

- We may resort to iterative approaches →
Gradient Descent

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \gamma \nabla J(S; \mathbf{w}_{t-1})$$

$\mathbf{w}_0 = 0$ Or with random values

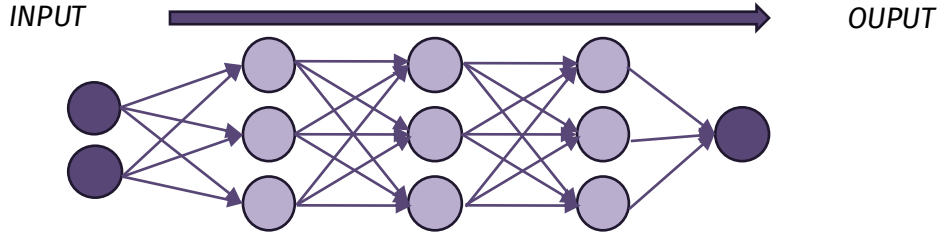
- We need to compute the gradient of a possibly very complex function!



Training a Neural Network

- Initialize the network with random weights $\mathbf{w}_0 \sim \mathcal{N}(0, \sigma^2)$
- Until convergence:
 - Compute the gradient $\nabla J(\mathbf{w}_{t-1})$
 - Update the weight $\mathbf{w}_t = \mathbf{w}_{t-1} - \gamma \nabla J(S; \mathbf{w}_{t-1})$
- Return the weights

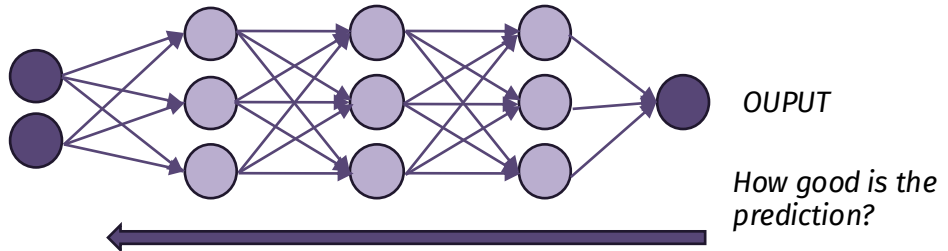
One network, two phases



*GOAL: obtaining
the prediction*

- **Forward propagation:** the input flows into the network and produces a cost
- **Backward propagation:** allows the info to flow back into the net to compute the gradient (during the optimization)

*GOAL: updating
the weight*



Training a DNN

Back-propagation

- Backpropagation (1960s) aims to minimize the cost function by adjusting the weights and biases of the networks
- The level of adjustment is determined by the gradients of the cost function with respect to those parameters.
- The goal of backpropagation is to compute the partial derivatives of the cost function with respect to any parameter in the network.

Training a DNN

Back-propagation

- A key ingredient of back-propagation is the chain rule of derivation
- Example: is F is a composite function such that

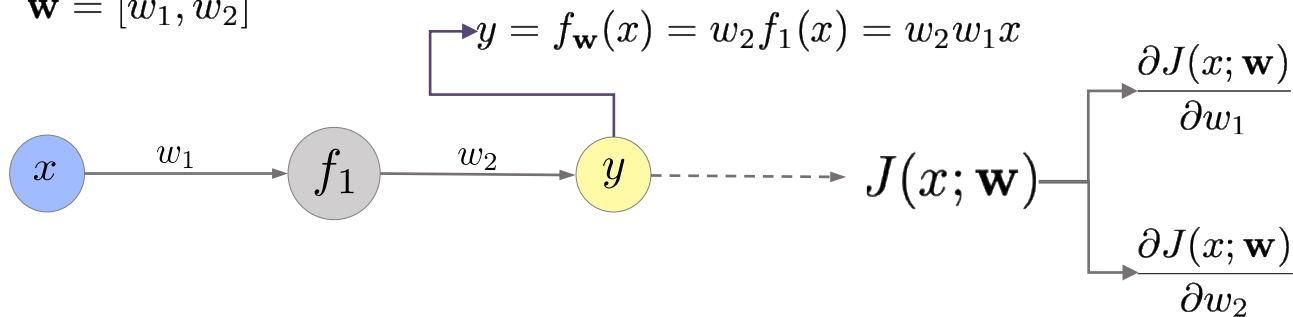
$$F = f \circ g \circ h \circ u \circ v$$

then

$$\frac{\partial F(x)}{\partial x} = \frac{\partial f(g(h(u(v(x))))))}{\partial x} = \frac{\partial f}{\partial g} \frac{\partial g}{\partial h} \frac{\partial h}{\partial u} \frac{\partial u}{\partial v} \frac{\partial v}{\partial x}$$

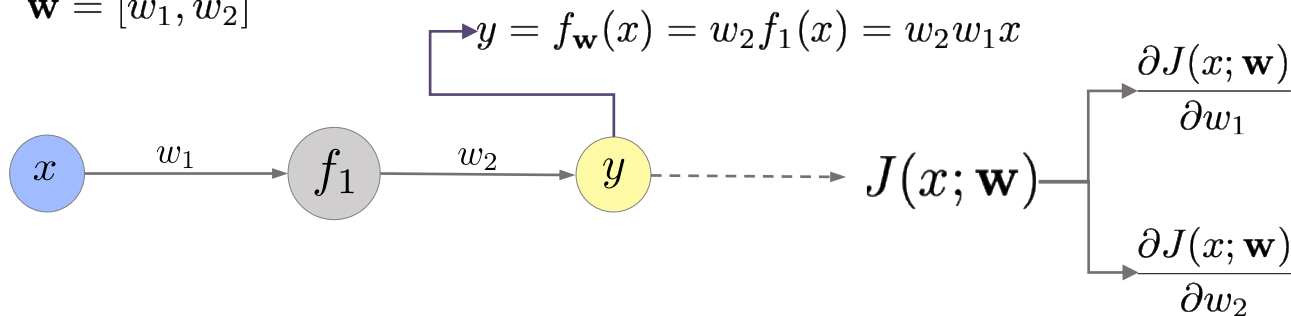
Backpropagation: example 1

$$\mathbf{w} = [w_1, w_2]$$



Backpropagation: example 1

$$\mathbf{w} = [w_1, w_2]$$

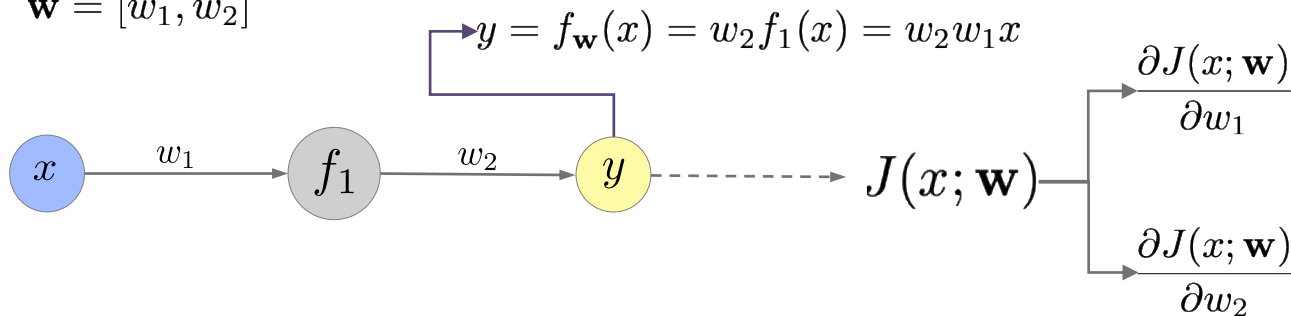


$$\frac{\partial J(x; \mathbf{w})}{\partial w_1}$$

$$\frac{\partial J(x, \mathbf{w})}{\partial w_1}$$

Backpropagation: example 1

$$\mathbf{w} = [w_1, w_2]$$

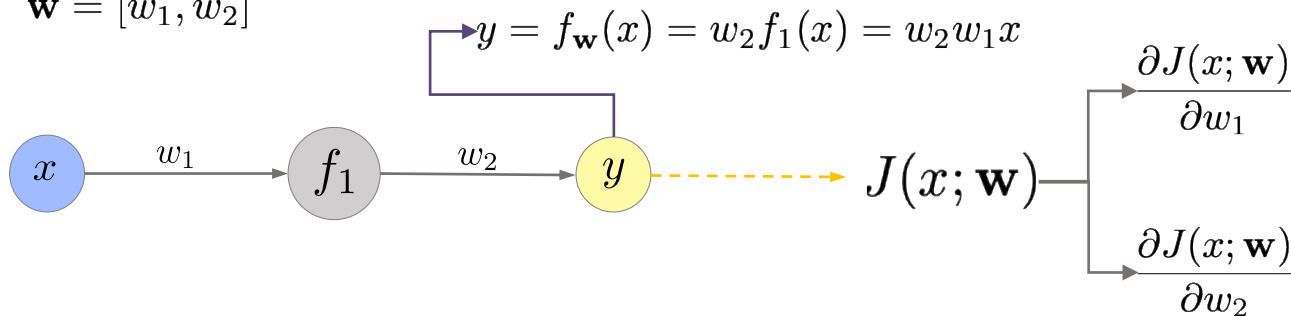


$$\frac{\partial J(x; \mathbf{w})}{\partial w_2} = \frac{\partial J(x; \mathbf{w})}{\partial f_{\mathbf{w}}(x)} \frac{\partial f_{\mathbf{w}}(x)}{\partial w_2}$$

$$\frac{\partial J(x; \mathbf{w})}{\partial w_1}$$

Backpropagation: example 1

$$\mathbf{w} = [w_1, w_2]$$

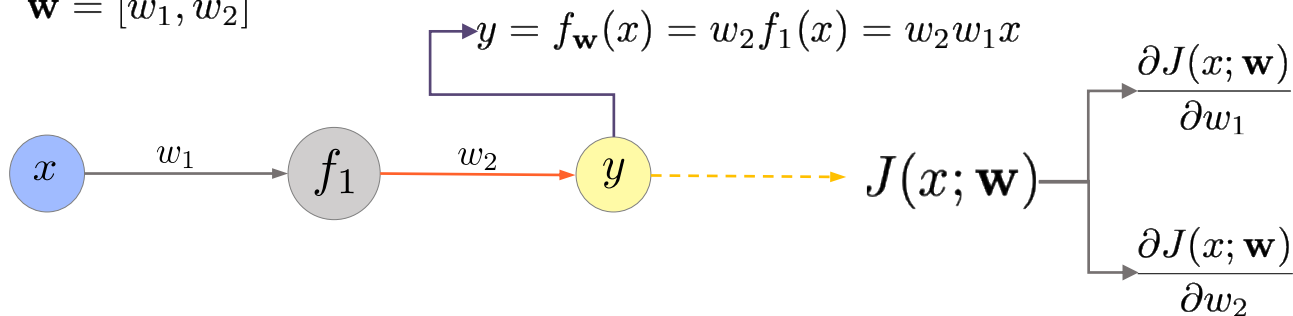


$$\frac{\partial J(x; \mathbf{w})}{\partial w_2} = \boxed{\frac{\partial J(x; \mathbf{w})}{\partial f_{\mathbf{w}}(x)}} \frac{\partial f_{\mathbf{w}}(x)}{\partial w_2}$$

$$\frac{\partial J(x; \mathbf{w})}{\partial w_1}$$

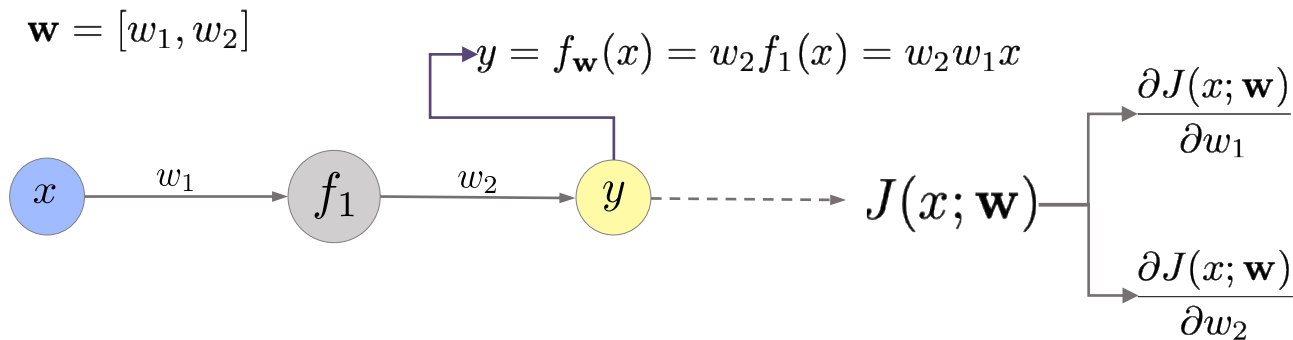
Backpropagation: example 1

$$\mathbf{w} = [w_1, w_2]$$



$$\frac{\partial J(x; \mathbf{w})}{\partial w_2} = \frac{\partial J(x; \mathbf{w})}{\partial f_{\mathbf{w}}(x)} \frac{\partial f_{\mathbf{w}}(x)}{\partial w_2}$$

$$\frac{\partial J(x; \mathbf{w})}{\partial w_1}$$

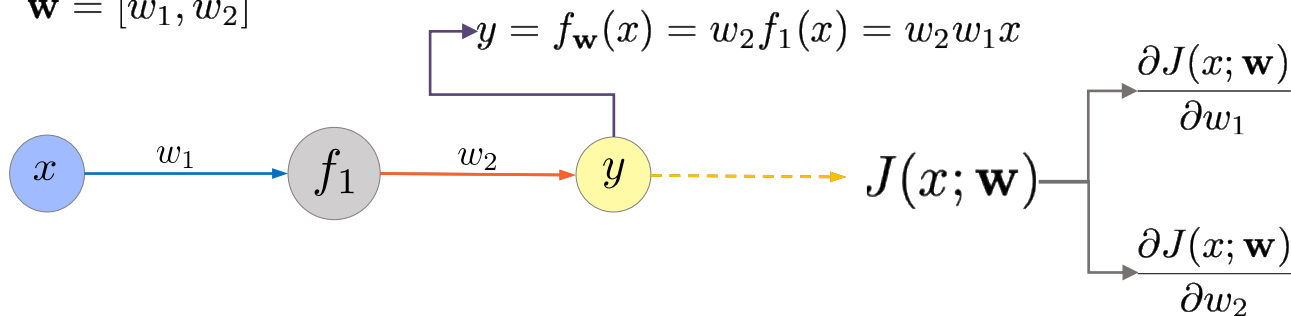


$$\frac{\partial J(x; \mathbf{w})}{\partial w_2} = \frac{\partial J(x; \mathbf{w})}{\partial f_{\mathbf{w}}(x)} \frac{\partial f_{\mathbf{w}}(x)}{\partial w_2}$$

$$\frac{\partial J(x; \mathbf{w})}{\partial w_1} = \frac{\partial J(x; \mathbf{w})}{\partial f_{\mathbf{w}}(x)} \frac{\partial f_{\mathbf{w}}(x)}{\partial f_1(x)} \frac{\partial f_1(x)}{\partial w_1}$$

Backpropagation: example 1

$$\mathbf{w} = [w_1, w_2]$$



$$\frac{\partial J(x; \mathbf{w})}{\partial w_2} = \frac{\partial J(x; \mathbf{w})}{\partial f_{\mathbf{w}}(x)} \frac{\partial f_{\mathbf{w}}(x)}{\partial w_2}$$

$$\frac{\partial J(x; \mathbf{w})}{\partial w_1} = \frac{\partial J(x; \mathbf{w})}{\partial f_{\mathbf{w}}(x)} \frac{\partial f_{\mathbf{w}}(x)}{\partial f_1(x)} \frac{\partial f_1(x)}{\partial w_1}$$

Backpropagation: example 2

- Let's consider the case where we have only one neuron, this time with a non-linearity
- Let's also assume we are using the square loss

$$\mathbf{x}^i \in X$$

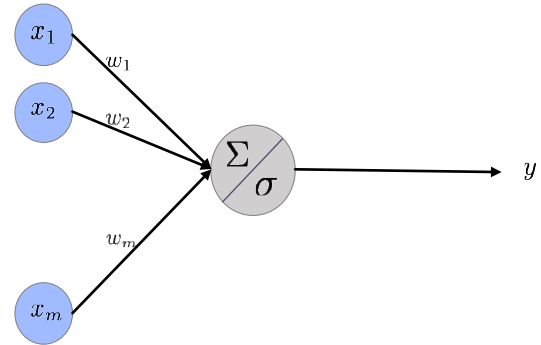
$$y^i \in Y$$

$$\text{Training set } S = \{\mathbf{x}^i, y^i\}_{i=1}^n$$

$$J(S; \mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y^i - f_{\mathbf{w}}(\mathbf{x}^i))^2 =$$

$$= \frac{1}{n} \sum_{i=1}^n (y^i - f_{\sigma}(\mathbf{w}^T \mathbf{x}^i))^2 =$$

$$= \frac{1}{n} \sum_{i=1}^n (y^i - \sigma(\mathbf{w}^T \mathbf{x}^i))^2$$



Backpropagation: example 2

- Let's consider the case where we have only one neuron, this time with a non-linearity
- Let's also assume we are using the square loss

$$\mathbf{x}^i \in X$$

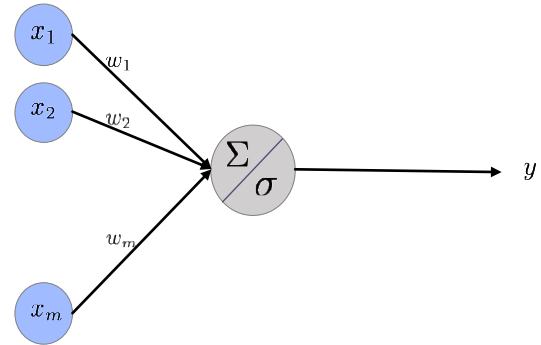
$$y^i \in Y$$

Training set $S = \{\mathbf{x}^i, y^i\}_{i=1}^n$

$$J(S; \mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y^i - f_{\mathbf{w}}(\mathbf{x}^i))^2 =$$

$$= \frac{1}{n} \sum_{i=1}^n (y^i - f_{\sigma}(\mathbf{w}^T \mathbf{x}^i))^2 =$$

$$= \frac{1}{n} \sum_{i=1}^n (y^i - \sigma(\mathbf{w}^T \mathbf{x}^i))^2$$



Backpropagation: example 2

$$J(S; \mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y^i - f_{\sigma}(\mathbf{w}^T \mathbf{x}^i))^2$$

Backpropagation: example 2

$$J(S; \mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y^i - f_{\sigma}(\mathbf{w}^T \mathbf{x}^i))^2$$

*For simplicity, we
consider the cost
function restricted
to sample i*

$$J((\mathbf{x}^i, y^i); \mathbf{w}) = (y^i - f_{\sigma}(\mathbf{w}^T \mathbf{x}^i))^2$$

Backpropagation: example 2

$$J(S; \mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y^i - f_{\sigma}(\mathbf{w}^T \mathbf{x}^i))^2$$

For simplicity, we consider the cost function restricted to sample i

$$J((\mathbf{x}^i, y^i); \mathbf{w}) = (y^i - f_{\sigma}(\mathbf{w}^T \mathbf{x}^i))^2$$

For the sake of compactness, we call it

$$J^i(\mathbf{w})$$

Backpropagation: example 2

$$J(S; \mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y^i - f_{\sigma}(\mathbf{w}^{\top} \mathbf{x}^i))^2$$

For simplicity, we consider the cost function restricted to sample i

$$J((\mathbf{x}^i, y^i); \mathbf{w}) = (y^i - f_{\sigma}(\mathbf{w}^{\top} \mathbf{x}^i))^2$$

For the sake of compactness, we call it

$$J^i(\mathbf{w})$$

We apply the chain rule of derivation

$$\frac{\partial J^i(\mathbf{w})}{\partial w_k} = \frac{\partial J^i(\mathbf{w})}{\partial f_{\sigma}(\mathbf{w}^{\top} \mathbf{x}^i)} \frac{\partial f_{\sigma}(\mathbf{w}^{\top} \mathbf{x}^i)}{\partial \mathbf{w}^{\top} \mathbf{x}^i} \frac{\partial \mathbf{w}^{\top} \mathbf{x}^i}{\partial w_k}$$

Backpropagation: example 2

$$\frac{\partial J^i(\mathbf{w})}{\partial w_k} = \frac{\partial J^i(\mathbf{w})}{\partial f_\sigma(\mathbf{w}^\top \mathbf{x}^i)} \frac{\partial f_\sigma(\mathbf{w}^\top \mathbf{x}^i)}{\partial \mathbf{w}^\top \mathbf{x}^i} \frac{\partial \mathbf{w}^\top \mathbf{x}^i}{\partial w_k}$$

Backpropagation: example 2

$$\frac{\partial J^i(\mathbf{w})}{\partial w_k} = \frac{\partial J^i(\mathbf{w})}{\partial f_\sigma(\mathbf{w}^\top \mathbf{x}^i)} \frac{\partial f_\sigma(\mathbf{w}^\top \mathbf{x}^i)}{\partial \mathbf{w}^\top \mathbf{x}^i} \boxed{\frac{\partial \mathbf{w}^\top \mathbf{x}^i}{\partial w_k}} \longrightarrow \boxed{x_k}$$

Backpropagation: example 2

$$\frac{\partial J^i(\mathbf{w})}{\partial w_k} = \frac{\partial J^i(\mathbf{w})}{\partial f_\sigma(\mathbf{w}^\top \mathbf{x}^i)} \frac{\partial f_\sigma(\mathbf{w}^\top \mathbf{x}^i)}{\partial \mathbf{w}^\top \mathbf{x}^i} \frac{\partial \mathbf{w}^\top \mathbf{x}^i}{\partial w_k} \rightarrow x_k$$

The diagram illustrates the backpropagation of gradients through the chain rule. The expression is broken down into three main components, each enclosed in a colored box:

- Orange box:** $\frac{\partial f_\sigma(\mathbf{w}^\top \mathbf{x}^i)}{\partial \mathbf{w}^\top \mathbf{x}^i}$ (The derivative of the activation function with respect to the weighted input).
- Blue box:** $\frac{\partial \mathbf{w}^\top \mathbf{x}^i}{\partial w_k}$ (The derivative of the weighted input with respect to the weight w_k).
- Blue box:** x_k (The input x_k).

Arrows indicate the flow of the gradient from the input x_k through the weighted input and the activation function to the final loss derivative. An additional arrow points from the orange box to a separate box containing the derivative of the activation function:

Orange box: $f'_\sigma(\mathbf{w}^\top \mathbf{x}^i)$

Backpropagation: example 2

$$\frac{\partial J^i(\mathbf{w})}{\partial w_k} = \frac{\partial J^i(\mathbf{w})}{\partial f_\sigma(\mathbf{w}^\top \mathbf{x}^i)} \frac{\partial f_\sigma(\mathbf{w}^\top \mathbf{x}^i)}{\partial \mathbf{w}^\top \mathbf{x}^i} \frac{\partial \mathbf{w}^\top \mathbf{x}^i}{\partial w_k} \rightarrow x_k^i$$

$$f'_\sigma(\mathbf{w}^\top \mathbf{x}^i)$$

$$\frac{\partial J^i(\mathbf{w})}{\partial f_\sigma(\mathbf{w}^T \mathbf{x}^i)} = \frac{\partial (y^i - f_\sigma(\mathbf{w}^T \mathbf{x}^i))^2}{\partial f_\sigma(\mathbf{w}^T \mathbf{x}^i)} = -2(y^i - f_\sigma(\mathbf{w}^T \mathbf{x}^i))$$

Backpropagation: example 2

$$\frac{\partial J^i(\mathbf{w})}{\partial w_k} = \frac{\partial J^i(\mathbf{w})}{\partial f_\sigma(\mathbf{w}^\top \mathbf{x}^i)} \frac{\partial f_\sigma(\mathbf{w}^\top \mathbf{x}^i)}{\partial \mathbf{w}^\top \mathbf{x}^i} \frac{\partial \mathbf{w}^\top \mathbf{x}^i}{\partial w_k} \rightarrow x_k^i$$

$$f'_\sigma(\mathbf{w}^\top \mathbf{x}^i)$$

$$\frac{\partial J^i(\mathbf{w})}{\partial f_\sigma(\mathbf{w}^\top \mathbf{x}^i)} = \frac{\partial (y^i - f_\sigma(\mathbf{w}^\top \mathbf{x}^i))^2}{\partial f_\sigma(\mathbf{w}^\top \mathbf{x}^i)} = -2(y^i - f_\sigma(\mathbf{w}^\top \mathbf{x}^i))$$

$$\frac{\partial J^i(\mathbf{w})}{\partial w_k} = -2(y^i - f_\sigma(\mathbf{w}^\top \mathbf{x}^i)) f'_\sigma(\mathbf{w}^\top \mathbf{x}^i) x_k^i$$

This was for a single unit and a single sample

What to do for using all the samples?

What to do when we have multiple units?

Training a [Deep] Neural Network

A parenthesis on Gradient Descent algorithms

$$\mathbf{w}^* = \arg \min_{\mathbf{w} \in \mathbb{R}^P} J(S; \mathbf{w})$$

$$\mathbf{w}_0 = 0$$

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \gamma \nabla J(S; \mathbf{w}_{t-1})$$

Training a [Deep] Neural Network

A parenthesis on Gradient Descent algorithms

$$\mathbf{w}^* = \arg \min_{\mathbf{w} \in \mathbb{R}^P} J(S; \mathbf{w})$$

$$\mathbf{w}_0 = 0$$

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \gamma \nabla J(S; \mathbf{w}_{t-1})$$

Batch Gradient Descent (GD)

Smoothly converging towards the optimum, but computationally demanding

Training a [Deep] Neural Network

A parenthesis on Gradient Descent algorithms

$$\mathbf{w}^* = \arg \min_{\mathbf{w} \in \mathbb{R}^P} J(S; \mathbf{w})$$

$$\mathbf{w}_0 = 0$$

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \gamma \nabla J(S; \mathbf{w}_{t-1})$$

Batch Gradient Descent (GD)

Smoothly converging towards the optimum, but computationally demanding


$$\mathbf{w}_t = \mathbf{w}_{t-1} - \gamma \nabla J(x^i; \mathbf{w}_{t-1})$$

Stochastic Gradient Descent (SGD)

*known to converge well in practice:
empirically, it provides a better
exploration of the space*

May require many iterations

Training a [Deep] Neural Network

A parenthesis on Gradient Descent algorithms

$$\mathbf{w}^* = \arg \min_{\mathbf{w} \in \mathbb{R}^P} J(S; \mathbf{w})$$

$$\mathbf{w}_0 = 0$$

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \gamma \nabla J(S; \mathbf{w}_{t-1})$$

Batch Gradient Descent (DG)

Smoothly converging towards the optimum, but computationally demanding

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \gamma \nabla J(x^i; \mathbf{w}_{t-1})$$

Stochastic Gradient Descent (SGD)

Known to converge well in practice: empirically, it provides a better exploration of the space

May require many iterations

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \gamma \nabla J(x^{[i:i+B]}; \mathbf{w}_{t-1})$$

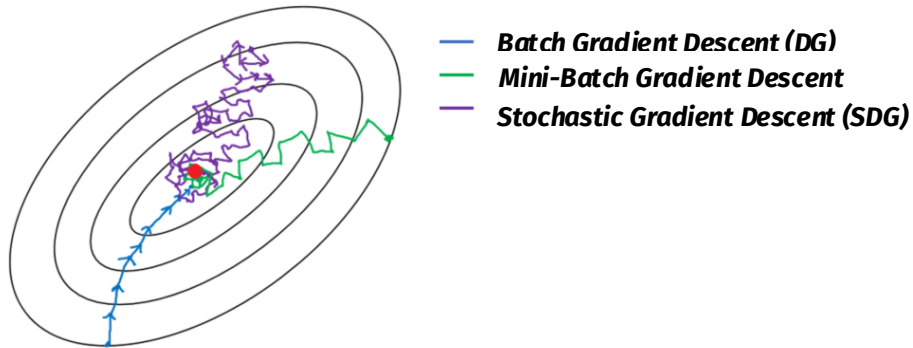
Mini-batch Gradient Descent

A good compromise

B is the mini-batch size (when B=1 you go back to SGD)

Training a [Deep] Neural Network

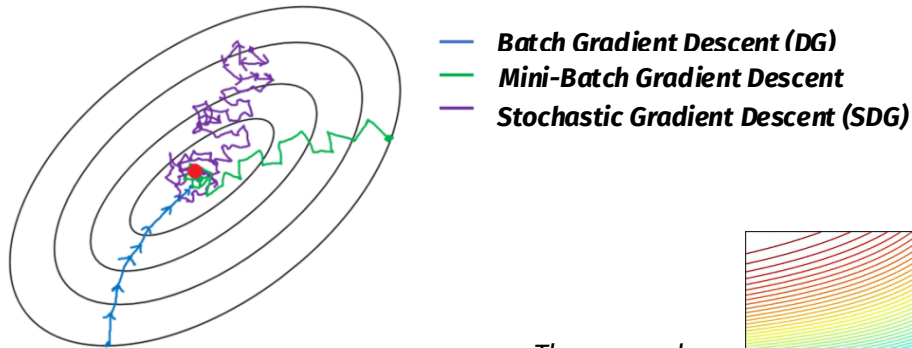
A parenthesis on Gradient Descent algorithms



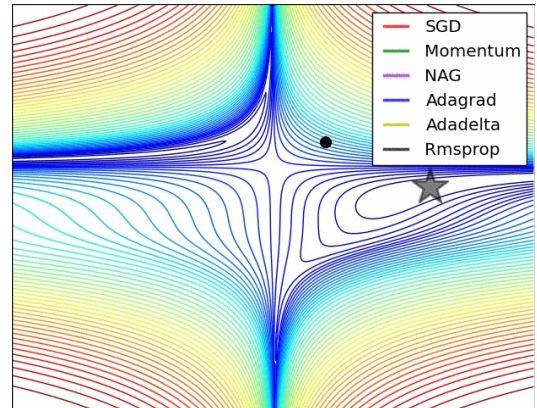
Picture from <https://towardsdatascience.com/gradient-descent-algorithm-and-its-variants-10f652806a3>

Training a [Deep] Neural Network

A parenthesis on Gradient Descent algorithms



There are also
alternative
optimization
strategies...



Picture from <https://towardsdatascience.com/gradient-descent-algorithm-and-its-variants-10f652806a3>

A note of the learning rate

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \gamma \nabla J(S; \mathbf{w}_{t-1})$$

$\mathbf{w}_0 = 0$

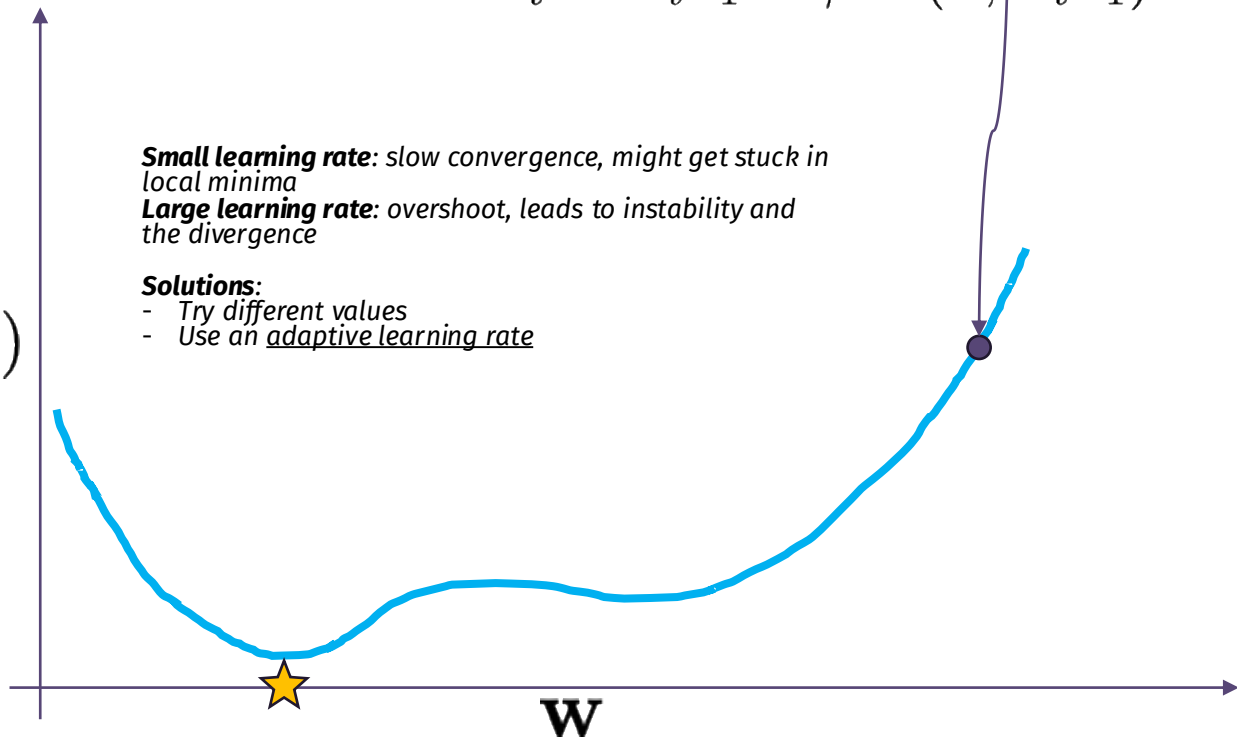
Small learning rate: slow convergence, might get stuck in local minima

Large learning rate: overshoot, leads to instability and the divergence

Solutions:

- Try different values
- Use an adaptive learning rate

$J(\mathbf{w})$



Example from https://introtodeeplearning.com/slides/6S191_MIT_DeepLearning_L1.pdf

Training a [Deep] Neural Network

A parenthesis on terminology

Given the complexity of deep models, it is necessary to go through the entire training set more than once at training time

One **epoch** is when the dataset is entirely passed forward and backwards through the architecture

The **[mini-]batch** size is the number of training samples in a mini-batch

An **iteration** is the number of batches needed to complete one epoch

→ Number of epochs and batch size are hyper-parameters of the model, usually set a priori

Training a [Deep] Neural Network

Model selection

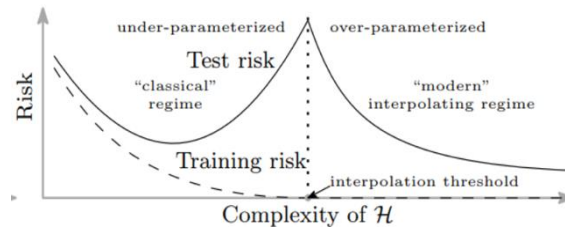
What does it mean to perform model selection with this family of methods?

Training a [Deep] Neural Network

Model selection

What does it mean to perform model selection with this family of methods?

In principle, one could apply the same protocol adopted for more classical methods



What's the complexity here?

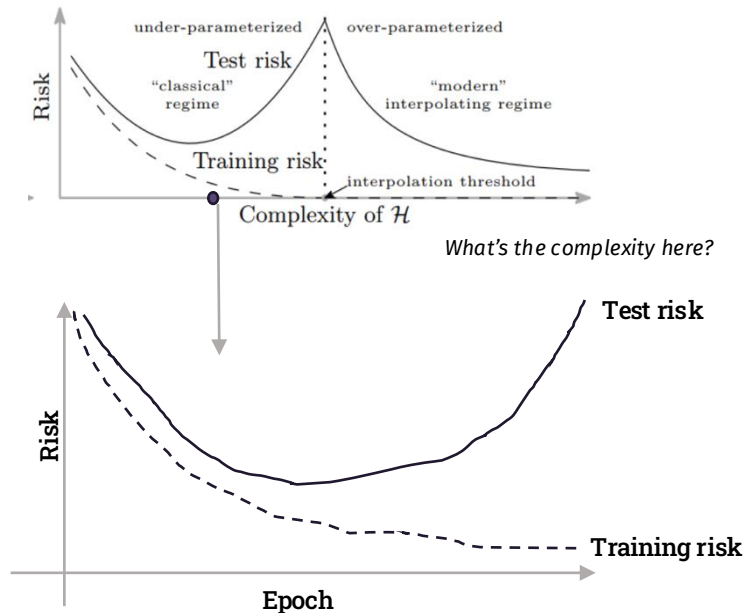
Training a [Deep] Neural Network

Model selection

What does it mean to perform model selection with this family of methods?

In principle, one could apply the same protocol adopted for more classical methods

In practice, we fix the model (i.e. a certain complexity) and, often, also the values of many (if not all) hyper-parameters; we reason on the training-validation error vs the number of epochs



Training a [Deep] Neural Network

How to prevent overfitting

- Classical approaches can still be adopted:
 - Adding regularization terms
 - Using Early Stopping criterion
- One alternative (popular) option: Dropout

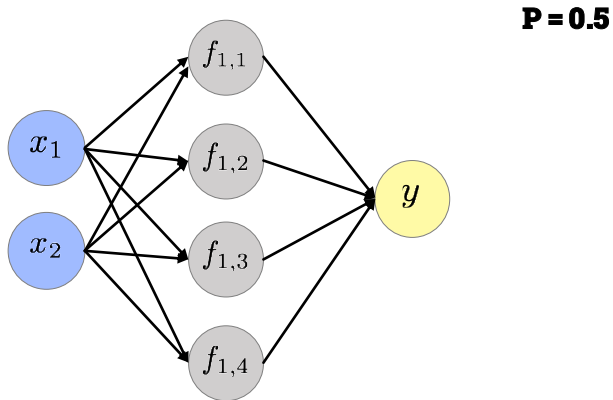
Training a [Deep] Neural Network

Dropout

- Overfitted DNN models tend to suffer from a problem of co-adaptation: models weights are adjusted co-linearly to learn the model training data too well... so the model doesn't generalize
- With limited training data weights likely become adapted to them, and the model doesn't generalize
- Bagging and Dropout are ways to break this co-adaptation

Training a [Deep] Neural Network

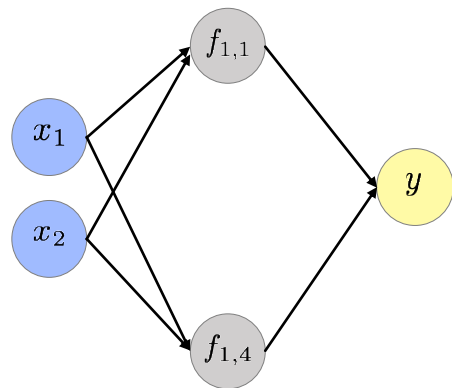
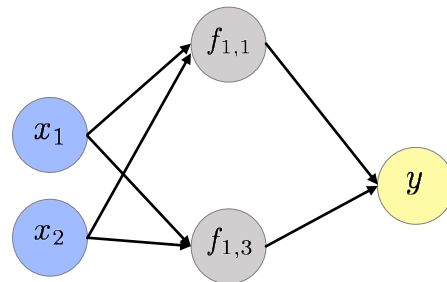
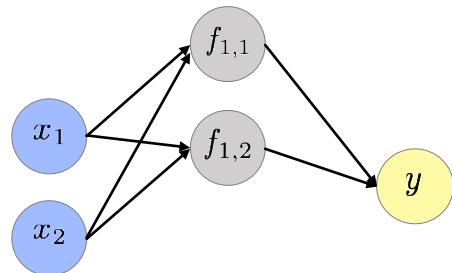
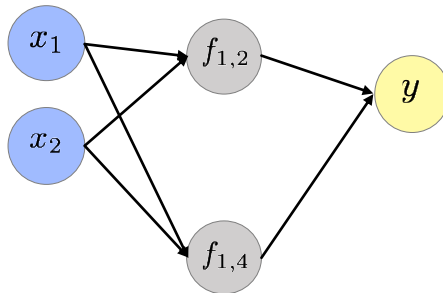
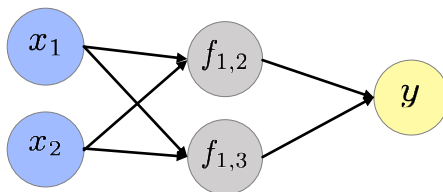
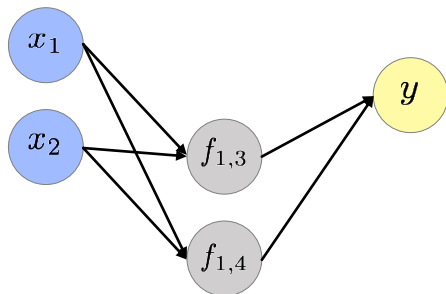
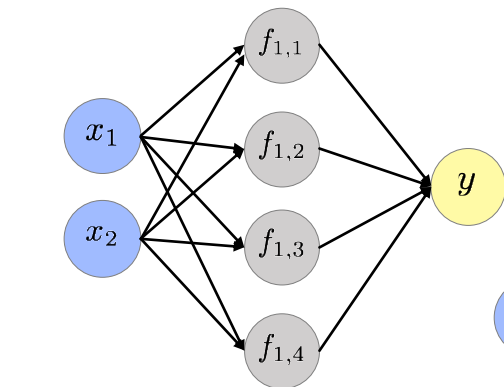
Bagging: intuition



Training a [Deep] Neural Network

Bagging: intuition

P = 0.5



Training a [Deep] Neural Network

Bagging and Dropout

With Bagging

- Each sub-network is trained and evaluated on each test sample
- The final prediction is given by the votes of all models

Bagging may be computationally very expensive

Dropout is a way to approximate the same behaviour

→ At each step of the optimisation, some fraction of weights are dropped out of each layer (=set to 0)

Wrapping up

- We introduced the concepts at the basis of Neural Networks
- We discussed how to train a Neural Network

Next class

We move to different tasks

- Variable selection
- Unsupervised Learning

UniGe

